

fmix Phase A: Structure, Rationale, and the Generation Benchmarks

2026-07-21

documents the phase-A machinery as of commits e38d4ac0, 8cb83033, 8748caea, b2f4d702 on
ssg-gen-mix-clean

1 Context: the two-phase mixing pipeline

Production mixing of a gadgetized circuit runs in two phases:

- **Phase A** (this document): churn at (roughly) constant, small size. The input is a gadgetized circuit; the goal is to erase the gadgetizer’s deterministic authorship — re-encode every local neighborhood, rotate the sharing frame, homogenize the syntactic texture — while the circuit is small and every operation is cheap.
- **Phase B**: anneal the DB channel away and grow by split transport to the final size. Fossil erosion scales with the pure-split growth ratio (R-law: ratio ≥ 3 –4 for $\sim 1\%$ -class residuals), so phase B wants to *start small* — which is phase A’s other job: deliver its dose without inflating the circuit first.

The tension that shapes everything below: the moves that re-encode most effectively also grow the circuit, and growth belongs to phase B. Phase A is therefore built around a **dose accounting** (how much re-encoding has each gate actually received?) and a **payment discipline** (spend growth only where it is provably the only way to buy the dose).

2 What phase A must accomplish

Three re-randomizations, each with its own meter:

1. **Local nonlinear re-encoding** — DB replacements substitute a window (2–5 gates) with a random equivalent implementation drawn from the frozen store. A single replacement is invisible outside its ~ 6 -wire span; security comes from *composition of overlapping rewrites*. This is the only lever against degree-2 leakage of the original computation (affine twists provably cannot touch it), which makes it the security-critical axis. Its meter is the **generation census** of §4.
2. **Frame erasure** — conjugation twists rotate the wire frame position-by-position. Meter: per-position twist coverage $\text{cov} = \text{twisted span}/\text{size}$, with the measured saturation law $256(1 - e^{-c/256})$ giving a target of $\text{cov} \approx 600\times$. Coverage per twist scales inversely with circuit size — the arithmetic reason the twist budget belongs in phase A.
3. **Texture homogenization** — the walk’s split/merge churn plus DB re-spelling converts gadget-template material into store-native equilibrium material and erodes origin fossils, serving the static (non-executing) distinguisher front.

3 The round scheduler

Each move is one round. Top-level coins, in order:

coin	round type	size effect
p_twist	one conjugation twist (type from <code>w_twist_*</code> ratios)	+2/+6 brackets
p_db_ingest	cheap ingest : Compressing-mode DB attempt, cheap-tier seed	never grows
p_db_hard	paid : MinGrow-mode DB attempt, hard-tier seed	minimum spelling; ledgered
p_db_eff	generic size-agnostic DB round (steered/annealed)	any
otherwise	the walk: contract (undo/merge/ <code>w_db</code>) or expand (cross/split/insert)	$\pm$ small

The thermostat holds size near `--target-size`; the steer multiplies the generic DB rate by $\sigma(-\text{excess}/\text{temp})$. An important measured consequence: a run sitting at or above target has its generic agnostic channel throttled to $p \approx 0.001$ — in that regime essentially all re-encoding flows through the non-growing and paid channels, which is exactly the discipline phase A wants.

4 Generation accounting

Every gate carries a **rewrite generation**:

- Input gates start at generation 0.
- A **DB replacement** stamps its products with the *upper median* of the outgoing window’s generations, plus 1 (median rounded up on even window sizes; `--gen-median-low` selects the lower median).
- A **split** (presplit, cross piece, fresh-split, unsum, twist case-split) stamps children with parent + 1 under the default *ratchet* rule; `--gen-split-inherit` makes children inherit the parent generation unchanged, so that only DB replacements raise generations.
- **Twist bracket packets** and **fresh insert pairs** are born-random material carrying no input structure: they get MAXGEN, higher than every real generation.
- Merges take the minimum of their parents; undos restore the recorded pre-split generations.

The **circuit generation** is the largest G such that at least 95% of *all* gates have generation $\geq G$ (the 5th-percentile gate generation, reported `G=`). The **dose stop** ends the run at the first report point where the below-target fraction over all gates is $\leq \text{--gen-stop-frac}$ ($0.05 =$ “the circuit has reached generation G ”) and the twist coverage target (`--twist-cov-stop`, if set) is met. The move budget becomes a ceiling: phase A runs exactly as long as the dose requires, which is the minimal-growth schedule.

5 Targeting: the laggard census and tiers

Uniform window seeding is a coupon-collector process — the last un-encoded gates absorb most of the moves. Instead, fmix maintains a census of **laggards** (gates below `--gen-target`), rebuilt every `--gen-rescan` moves and pruned lazily at draw time, and seeds DB windows from it with probability `--gen-bias`.

Each gate also carries a **miss counter** (reset on every splice product): laggard-seeded attempts that fail to consume their seed bump it. The counter partitions laggards into:

- **cheap tier** ($\text{miss} < \text{--gen-miss-budget}$): still worth trying to ingest with non-growing replacements;
- **hard tier**: the cheap channel has *proven* the gate has no non-growing spelling in its current context — only the paid channel touches it;
- **retired** ($\text{miss} \geq \text{--gen-giveup}$, optional): written off, reported $u=$, dropped from targeting (but still counted by the all-gates dose criterion).

6 Ingest-then-pay

The policy (“first ingest as many gates as possible with tight control over growth, then pay in growth only for those gates that cannot be ingested otherwise”) rests on a measured dichotomy:

- **Compress-only regimes leave a large stuck population.** With re-encoding restricted to non-growing replacements, $\sim 25\text{--}35\%$ of gates were never consumed by any splice — the type join was unambiguous: g57 gates 0.2% stuck, CNOTs 89%, non-g57 conjunctions 91%. A CNOT alone costs six g57s: windows containing one essentially never have a non-growing spelling.
- **Unrestricted any-length replacement reaches everything but overpays.** With the agnostic channel live, the stuck set collapsed to 3.5% — at the cost of $4.5\times$ growth.

Ingest-then-pay separates the two channels and gates the expensive one per-gate: `--p-db-ingest` rounds (run hot, ~ 0.5) use Compressing mode — non-growing replacements only, zero size risk — seeded on the cheap tier; `--p-db-hard` rounds (low rate, ~ 0.05) use **MinGrow** mode — uniform among the *shortest* equivalents, growing allowed — seeded on the hard tier only, never falling back to easy material, with every added gate ledgered (`paid=`).

The validating pilot (raw CNOT-gadgetized cg1, 48,165 gates, generation target 2): dose stop fired at 900k moves with $2.12\times$ growth versus $4.5\times$ for unrestricted agnostic mixing at the same dose; `paid` = 52,089 accounted for essentially all growth; the paid channel hit on 94% of its attempts; exactly one gate in the run was retired as unreachable. The store can spell virtually everything — the earlier “unreachable population” was entirely the non-growing constraint, and the miss-budget machinery locates the gates that need payment and pays each one at its minimum spelling.

7 The generation benchmarks

Setup. Input: a gadgetized sliced sandwich on $n = 128$ (two-sided keyed slicing on 256 wires, CNOT-gadgetized to 512 wires): 99,016 gates. Recipe: `target-size` 100k, `p-db-ingest` 0.5, `p-db-hard` 0.05, `miss-budget` 6, `giveup` 0, `gen-bias` 0.9, `p_twist` 0.0016 (neg-only, `twist-min-len` 256), windows [2, 5] with largest-first prefix descent, mixed sampler, `ctrl-cap` 2, $\text{degree} \leq 9$, $\text{span} \leq 30$, term caps 1024/2048, `stop-frac` 0.05, seed 7. Question: **how many gates does phase A end with, as a function of the generation target G ?**

Ratchet split rule (children = parent + 1):

target G	stop at	final gates	inflation	paid
2	1.3M moves	177,390	$1.79\times$	84,464
3	1.4M	180,458	$1.82\times$	87,747
5	1.5M	182,742	$1.85\times$	90,307
10	1.7M	186,632	$1.88\times$	94,984

Inherit split rule (`--gen-split-inherit`; only DB replacements raise generations):

target G	stop at	final gates	inflation	paid
2	1.4M moves	181,377	$1.83\times$	88,807
3	1.4M	180,139	$1.82\times$	87,553
5	1.6M	185,783	$1.88\times$	94,263
10	1.7M	186,391	$1.88\times$	95,423
20	2.0M	193,405	$1.95\times$	104,612
30	2.3M	199,288	$2.01\times$	112,878
40	2.7M	210,934	$2.13\times$	129,236
50	3.1M	221,234	$2.23\times$	142,279

Lower-median variant (`--gen-median-low`, `inherit`, $G = 5$): 188,127 gates at 1.7M moves — versus 185,783 at 1.6M for the upper median.

8 Interpretation

1. **The first generation is the whole entry price.** Reaching $G = 2$ costs $\sim 1.8\times$ — the coupon-collector sweep that touches every position once. The `paid=` ledger (85–95k gates in every run) accounts for essentially all growth: what phase A buys with size is the base coverage of the material that has no free spelling, once.
2. **The curve is flat through $G \approx 10$, then gently linear.** From $G = 10$ to $G = 50$ the cost rises by a steady ~ 870 gates ($\sim 0.9\%$ of the input) and $\sim 35k$ moves per additional level: $1.88\times$ at $G = 10 \rightarrow 2.23\times$ at $G = 50$. The mid-run shape shows why: the bulk of the circuit rides the median rule’s compounding (one re-splice lifts a whole neighborhood a level), while the linear tail is the targeted machinery grinding the bottom 5% level by level at the splice cadence.
3. **The stamp-rule choices barely matter — because targeting neutralizes them.** Ratchet vs. inherit: differences at the noise level (the split ratchet is a minor accelerant; the median rule dominates). Upper vs. lower median (the min on 2-gate windows, the most common splice): $\sim 1\%$ in size, $\sim 100k$ moves. Whatever gates a stingier rule leaves behind simply *become* the laggard seed pool and get ground down at the same cadence. The cost curve is set by the targeting + dose-stop machinery; the stamp semantics only choose where the flat regime ends. (The one semantics that would change the picture qualitatively is min-over-window: every straggler drags its whole window down, and the cost of G becomes $\sim$ linear from the start.)
4. **Practical consequence: choose G by security need, not budget.** The marginal cost of generation depth is so low beyond the first sweep that $G = 20$ –30 is affordable insurance.

The right value should come from the still-open dose-response question — how degree-2 reconstruction of the original computation decays with re-encoding depth — not from size pressure.

5. **Twist coverage rides along.** The benchmark runs accumulated $\text{cov} = 178\text{--}374\times$ incidentally; the $\sim 600\times$ saturation target needs either the longer high- G runs or a modestly raised `p_twist`, and can be armed as a joint stopping condition (`--twist-cov-stop`).

9 Open threads

- **Generation $\leftrightarrow$ leakage dose-response:** measure degree-2 reconstruction of the original circuit against generation depth at a tractable size; use it to pick the production G .
- **Dose-then-shrink:** after the dose stop, a compress-heavy epoch (`w_db 1`, `p_db 0`) to bring the size back toward the phase-B switch size. Generations survive compression by construction (merges take the min, compressing splices still stamp `median+1`).
- **Pay-ordering refinement:** hold the paid channel until the cheap tier is nearly empty, and/or let cheap rounds take first crack at paid products — both would shave the $\sim G$ -fold re-payment of hard cores.
- **Static positional-uniformity instrument** (sliding-window gate statistics along the circuit) — the other distinguisher front, still to be built.